

CS383 - Software Engineering

Software Implementation (II)

Outline

- Code smells
- Code refactoring

Bad Smells in Code

- by Kent Beck and Martin Fowler

If it stinks, change it.

- Signs that you need to refactor

Bad Smells in Code (cont..)

Duplicated Code	Primitive Obsession	Inappropriate Intimacy
Long Method	Switch Statements	Alternative Classes with Different Interfaces
Large Class	Parallel Inheritance Hierarchies	Incomplete Library Class
Long Parameter List	Lazy Class	Data Class
Divergent Change	Speculative Generality	Refused Bequest
Shotgun Surgery	Temporary Field	Comments
Feature Envy	Message Chains	
Data Clumps	Middle Man	

Duplicated Code

- The same code structure in more than one place.
- E.g.: have the same expression in two methods of the same class.
- Solution: perform Extract Method and invoke the code from both places.

Long Method

- The longer a procedure is, the more difficult it is to understand.
- E.g.: Find a very long method
- Solution: Perform Extract Method and improve responsibility distribution.

Large Class

- When a class is trying to do too much
 - Solution: Extract Class or Extract Subclass
- When a class has too many instance variables, duplicated code cannot be far behind.
 - Solution: Remove duplication using Extract Method

Long Parameter List

- Long parameter lists are hard to understand
- In OO tend to make parameter list shorter
- Problem: finding a method that has a very long parameter list
- E.g.: `public void person ( String FName, String Lname, int Age, boolean Gender, char Status) {.....}`
- Solution: Replace parameters with Methods or Introduce parameter object

Divergent change

- Divergent change occurs when one class is commonly changed in different ways for different reasons
- If for example you have to change 4 different methods every time the database changes you might have a situation where two objects are better than one
- To clean this up you identify everything that changes for a particular cause and use Extract Class to put them all together

Data Clumps

- If multiple data appears together in lots of places
- E.g. fields in a couple of classes (1), parameters in many method signatures (2).
- Solution (1): use Extract Class to turn the clumps into an object
- Solution (2): use Introduce Parameter Object or Preserve Whole Object to slim them down

Switch Statements

- The problem with switch statements is essentially that of duplication
- Adding a new case requires changing all the switches
- OO of polymorphism gives you an elegant way to deal with this problem.
- **Solutions:** Use Extract Method to extract the switch statement
- Use Move Method to get it onto the class where the polymorphism is needed.

Comments

- Comments aren't a bad smell; indeed they are a sweet smell.
- Comments lead us to bad code
- Solutions: if you need a comment to explain what a block of code does, try Extract Method.
- If the method is already extracted but you still need a comment to explain what it does, use Rename Method.
- A comment is a good place to say why you did something.

Code refactoring

- Definition: refactoring modifies software to improve its readability, maintainability, and extensibility without changing what it actually does.
- External behavior does NOT change
- Only internal structure is improved
- Refactoring is a very important element in Agile development
 - Makes agile methodology better than classic incremental development process model

Code refactoring (cont.d)

- The goal of refactoring is NOT to add new functionality
- The goal of refactoring is to make code easier to maintain in the future

Why we do refactoring?

- Improve software design
- Enable understandability
- Help finding bugs

Code refactoring (cont.d)

- The goal of refactoring is NOT to add new functionality
- The goal of refactoring is to make code easier to maintain in the future

Low-level refactoring

- **Names:**

- Renaming (methods, variables)

- **Procedures:**

- Extracting code into a method
- Extracting common functionality into a module/method/etc
- Inlining a method/procedure
- Changing method signatures

Low-level refactoring

- **Reordering:**
- Splitting one method into several to improve cohesion and readability (by reducing its size)
- Putting statements that semantically belong together near each other

IDE support for low-level refactoring

Eclipse / Visual Studio support:

- variable / method / class renaming
- method or constant extraction
- extraction of redundant code snippets
- method signature change
- extraction of an interface from a type
- method inlining
- providing warnings about method invocations with inconsistent parameters

High-level refactoring

Deep implementation and design changes

- Refactoring to design patterns
- Performance optimization
- Clarifying a statement that has evolved over time or is unclear

Refactoring plan (1/2)

- ❑ Write unit tests that verify the code's external correctness.
 - They should pass on the current poorly designed code
 - Having unit tests helps make sure any refactor doesn't break existing behavior (regressions).

- ❑ Analyze the code to decide the risk and benefit of refactoring.
 - If it is too risky, not enough time remains, or the refactor will not produce enough benefit to the project, don't do it.

Refactoring plan (2/2)

- ❑ Refactor the code.
 - Some tests may break. Fix the bugs.
- ❑ Code review the changes
- ❑ Check in your refactored code
 - Keep each refactoring small; refactor one unit at a time.
 - Your checkin should contain only your refactor.
 - Your checkin should not contain other changes such as new features, fixes to unrelated bugs, and other tweaks.

Refactoring Simple Example

- Move a method
- Motivation: a method is, or will be, using or used by more features of another class than the class on which it is defined.
- Technique: Create a new method with a similar body in the class it uses most. Either turn the old method into a simple delegation, or remove it altogether.

Refactoring: Complex Example

- Introduce Null Object
- Motivation: You have many checks for null
- Technique: Replace the null value with a null object.

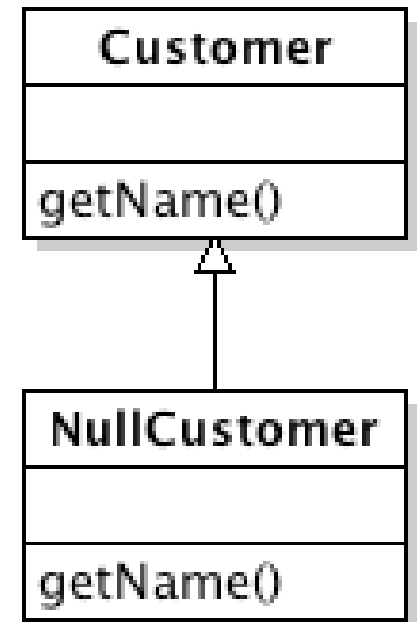
```
Customer customer = findCustomer(...);  
...  
    if (customer == null) {  
        name = "occupant"  
    } else {  
        name = customer.getName()  
    }  
  
if (customer == null) {  
...  
}
```

Refactoring: Complex Example

```
public class NullCustomer extends Customer{  
    public String getName() {  
        return "occupant";  
    }  
}
```

// -----

```
Customer c = findCustomer(...);  
name = c.getName();
```



- Completely eliminated the if statement by replacing checks for null with a null object that does the right thing for “null” values.

Refactoring Example: Replace Conditional with Polymorphism

- Motivation: You have a conditional that chooses different behavior depending on the type of an object.
- Technique: Move each leg of the conditional to an overriding method in a subclass. Make the original method an abstract.

```
double getSpeed() {  
    switch (_type) {  
        case EUROPEAN:  
            return getBaseSpeed();  
        case AFRICAN:  
            return getBaseSpeed() - getLoadFactor() * _numberOfCoconuts;  
        case NORWEGIAN_BLUE:  
            return (_isNailed) ? 0 : getBaseSpeed(_voltage);  
    }  
    throw new RuntimeException ("Should be unreachable");  
}
```

Refactoring Example: Replace Conditional with Polymorphism

